

Testing

Portfolio evidence: how I applied the testing workshop to LifeForest

Learning outcome claim

I applied testing in LifeForest by choosing high-value and high-risk areas to test, writing automated backend unit and integration tests, using mocks and an H2 test database, checking frontend quality with linting and TypeScript, and automating the process in GitHub Actions.

What Testing Means

Testing validates that software works as intended and that important changes do not break existing behaviour. It is about quality and trust: fewer bugs, more confidence, and stronger proof that the project works.

For LifeForest, this matters because the project combines a mobile frontend, Spring Boot API, database behaviour, authentication, routines, tasks, focus sessions, reflections, analytics, achievements, and forest progress.

My Testing Strategy

- Focused automated tests on high-value backend logic and API behaviour.
- Used the test pyramid: many unit/service tests plus integration/controller tests for combined behaviour.
- Used mocks to isolate service logic and test edge cases quickly.
- Used H2 with create-drop settings for database-backed integration tests.
- Used CI so tests and quality checks run automatically after changes.

Evidence From LifeForest

Testing idea	How I applied it	Evidence location	Why it matters
Unit tests	Service tests validate business logic with JUnit and Mockito.	backend/src/test/java	Fast feedback on core logic.
Mocking	Repository dependencies are mocked to control success, empty data, and error paths.	TaskServiceTest, AnalyticsServiceTest	Isolated and repeatable tests.
Integration tests	Controller integration tests use Spring Boot, MockMvc, repositories, transactions, and H2.	TreeControllerIntegrationTest and others	API and persistence checked together.
Regression testing	17 backend test files with 52 test methods can be rerun after changes.	backend/src/test/java	Old behaviour is protected.
Frontend checks	Expo linting and TypeScript type checking are available.	frontend/package.json	Catches UI mistakes early.

Testing idea	How I applied it	Evidence location	Why it matters
Automated process	GitHub Actions builds, tests, checks dependencies, lints, typechecks, and audits.	.github/workflows/ci.yml	Checks run consistently.
Manual testing	Login, tasks, routines, focus sessions, reflections, achievements, analytics, and forest progress were checked manually.	App behaviour and notes	Confirms user-facing flow.

Examples Of Test Quality

TaskServiceTest checks task trimming, saving, routine linking, and delete side effects.

AnalyticsServiceTest checks normal calculations, empty data, and unknown-user errors.

TreeControllerIntegrationTest verifies user-specific forest data through the API and database layer.

Reflection

The workshop changed my approach from 'I ran it and it worked' to a repeatable process. I now use automated backend tests, frontend static checks, and CI automation to catch problems earlier.

The strongest current evidence is backend service and controller coverage. The main next step is adding automated frontend tests and end-to-end tests for the most important user flows.

Next Improvements

- Add frontend unit tests with Jest or Vitest for components and hooks.
- Add end-to-end tests with Playwright, Cypress, or Detox for important flows.
- Add more security-focused tests for authentication, authorization, invalid JWTs, and user ownership rules.
- Keep screenshots or CI run links in Portflow as extra proof next to this document.